Crime Count Forecasting per District

IT462

Student Name	IDs	Email
Raghad Almutairi	443200793	443200793@student.ksu.edu.sa
Sarah Alruwayte	444200758	444200758@student.ksu.edu.sa
Norah Alfaheed	444200779	444200779@student.ksu.edu.sa
Atheer Budie	444200894	444200894@student.ksu.edu.sa

Section: 2766
Supervised by: Dr. AFSHAN JAFRI

List of Contents

1. Introduction *5*

2. Problem formulation *5*

3. Data preparation *6*

3.1 dataset overview 6

3.2 feature engineering 8

3.3 train/test split 10

3.4 standardscaler 10

4. Baseline model *11*

5. Full feature set models (35 features) *13*

5.1 linear regression 13

5.2 decision tree regressor 15

5.3 random forest regressor 16

5.4 model comparison — full features 19

5.5 feature importance — full random forest 20

6. Reduced feature set models (4 features) *22*

6.1 motivation for reduced feature set 22

6.2 model results — reduced features 24

6.3 feature importance — reduced random forest 25

6.4 full comparison table 26

7. Analysis and interpretation *27*

7.1 model performance analysis 27

7.2 feature importance insights 27

7.3 limitations and future work 28

List of Tables

Table 1: target variable distribution 6
Table 2: feature sets used in this phase 8
Table 3: train/test split statistics 10
Table 4: baseline model metrics 12
Table 5: linear regression metrics (full features) 14
Table 6: decision tree metrics (full features) 16
Table 7: random forest metrics (full features) 18
Table 8: model comparison — full feature set 19
Table 9: top 15 feature importances — full random forest 21
Table 10: model comparison — reduced feature set 24
Table 11: feature importances — reduced random forest 25
Table 12: full model comparison across all configurations 26

List of Figures

Figure 1: spark shell output showing step 1 – dataset load 7
Figure 2: spark shell output showing step 2 – data quality check 7
Figure 3: spark shell output showing step 3 – feature definition 9
Figure 4: spark shell output showing step 4 – feature assembly 9
Figure 5: spark shell output showing step 5 – train/test split 10
Figure 6: spark shell output showing step 6 – standardscaler 11
Figure 7: spark shell output showing step 7 – baseline metrics 12
Figure 8: spark shell output showing step 8 – linear regression 14
Figure 9: spark shell output showing step 9 – decision tree 16
Figure 10: spark shell output showing step 10 – random forest 18
Figure 11: spark shell output showing step 11 – model comparison 19
Figure 12: spark shell output showing step 12 – feature importances 20
Figure 13: spark shell output showing step 14/15 – reduced pipeline 23
Figure 14: spark shell output showing step 17 – reduced feature importance 25
Figure 15: spark shell output showing step 16 – full comparison table 26

1. Introduction

This phase applies Apache Spark MLlib to train and evaluate regression models for the Chicago district-level crime count forecasting problem. Three regression algorithms are trained and compared against a mean-prediction baseline: Linear Regression, Decision Tree Regressor, and Random Forest Regressor.

The initial feature set included all 35 available columns, with the expectation that including individual crime-type columns would produce a stronger model. However, evaluation revealed that these 31 columns collectively sum to the target variable, creating a mathematical identity that inflates Linear Regression performance without genuine forecasting value. A second evaluation using only the 4 temporal, spatial, and lag features was therefore conducted to obtain an honest measure of the model's true forecasting ability. Both configurations are documented and compared in this phase.

2. Problem Formulation

The forecasting problem is defined as follows:

Element	Description
Target variable	total_crime_count: total weekly crimes per district
Prediction unit	One prediction per district per week
Time period	2021–2024 (4 years, weekly granularity)
Problem type	Regression: predicting a continuous count value
Evaluation	RMSE, MAE, R^2 on held-out test set (30%)
Baseline	Mean prediction: predicts training mean for every week

The target variable total_crime_count ranges from 1 to 674 weekly crimes across all districts, with a mean of 204.77 and standard deviation of 66.86. A model that simply predicts the mean achieves RMSE = 68.61 and $R^2 \approx 0$, this is the performance floor all trained models must exceed to demonstrate genuine learning.

3. Data Preparation

3.1 Dataset Overview

The preprocessed dataset contains 4,644 aggregated weekly rows representing crime activity across 23 Chicago districts over 2021–2024. Each row represents one district-week combination with 38 columns: district identifier, week_start timestamp, 31 individual crime-type count columns (pivoted from primary_type), total_crime_count, month, week_no, district_indexed, and prev_week_total.

Table 1: Target variable distribution

Metric	Value
Minimum	1 crime per week
Maximum	674 crimes per week
Mean	204.77 crimes per week
Standard Deviation	66.86 crimes per week
Total rows	4,644 (23 districts × ~209 weeks)
Null values in key columns	0

```
     | // ========================================
     | // STEP 1: Load Preprocessed Dataset
     | // ========================================
     |
     | import org.apache.spark.sql.functions._
     | import org.apache.spark.ml.feature.{VectorAssembler, StandardScaler}
     | import org.apache.spark.ml.regression.{LinearRegression, DecisionTreeRegressor, RandomForestRegressor}
     | import org.apache.spark.ml.evaluation.RegressionEvaluator
     |
     | val preprocessedDF = spark.read
     |   .option("header", "true")
     |   .option("inferSchema", "true")
     |   .csv("/Users/raghadfares/Desktop/BigData/preprocessed_Data.csv")
     |
     | println(s"Preprocessed dataset rows: ${preprocessedDF.count()}")
     | println(s"Preprocessed dataset columns: ${preprocessedDF.columns.length}")
     | preprocessedDF.printSchema()
Preprocessed dataset rows: 4644
Preprocessed dataset columns: 38
root
 |-- district: integer (nullable = true)
 |-- week_start: timestamp (nullable = true)
 |-- ARSON: integer (nullable = true)
 |-- ASSAULT: integer (nullable = true)
 |-- BATTERY: integer (nullable = true)
 |-- BURGLARY: integer (nullable = true)
 |-- CONCEALED CARRY LICENSE VIOLATION: integer (nullable = true)
 |-- CRIMINAL DAMAGE: integer (nullable = true)
 |-- CRIMINAL SEXUAL ASSAULT: integer (nullable = true)
 |-- CRIMINAL TRESPASS: integer (nullable = true)
 |-- DECEPTIVE PRACTICE: integer (nullable = true)
 |-- GAMBLING: integer (nullable = true)
 |-- HOMICIDE: integer (nullable = true)
 |-- HUMAN TRAFFICKING: integer (nullable = true)
 |-- INTERFERENCE WITH PUBLIC OFFICER: integer (nullable = true)
 |-- INTIMIDATION: integer (nullable = true)
 |-- KIDNAPPING: integer (nullable = true)
 |-- LIQUOR LAW VIOLATION: integer (nullable = true)
 |-- MOTOR VEHICLE THEFT: integer (nullable = true)
 |-- NARCOTICS: integer (nullable = true)
 |-- NON-CRIMINAL: integer (nullable = true)
 |-- OBSCENITY: integer (nullable = true)
 |-- OFFENSE INVOLVING CHILDREN: integer (nullable = true)
 |-- OTHER NARCOTIC VIOLATION: integer (nullable = true)
 |-- OTHER OFFENSE: integer (nullable = true)
 |-- PROSTITUTION: integer (nullable = true)
 |-- PUBLIC INDECENCY: integer (nullable = true)
 |-- PUBLIC PEACE VIOLATION: integer (nullable = true)
 |-- ROBBERY: integer (nullable = true)
 |-- SEX OFFENSE: integer (nullable = true)
 |-- STALKING: integer (nullable = true)
 |-- THEFT: integer (nullable = true)
 |-- WEAPONS VIOLATION: integer (nullable = true)
 |-- total_crime_count: integer (nullable = true)
 |-- month: integer (nullable = true)
 |-- week_no: integer (nullable = true)
 |-- district_indexed: double (nullable = true)
 |-- prev_week_total: integer (nullable = true)
```

Figure 1: Spark shell output showing Step 1 – dataset load

```
scala> // ============================================
     | // STEP 2: Verify Data Quality Before ML
     | // ============================================
     |
     | println("Null counts in key columns:")
     | preprocessedDF.select(
     |   preprocessedDF.columns.filter(c =>
     |     Seq("total_crime_count","prev_week_total",
     |         "month","week_no","district_indexed").contains(c)
     |   ).map(c => sum(col(c).isNull.cast("int")).alias(c)): _*
     | ).show()
     |
     | preprocessedDF.select("total_crime_count")
     |   .summary("min", "max", "mean", "stddev")
     |   .show()
warning: 1 deprecation (since 2.13.0); for details, enable `:setting -deprecation` or `:replay -deprecation`
Null counts in key columns:
+-----------------+-----+-------+----------------+---------------+
|total_crime_count|month|week_no|district_indexed|prev_week_total|
+-----------------+-----+-------+----------------+---------------+
|                0|    0|      0|               0|              0|
+-----------------+-----+-------+----------------+---------------+

+-------+------------------+
|summary| total_crime_count|
+-------+------------------+
|    min|                 1|
|    max|               674|
|   mean|204.76614987080103|
| stddev|  66.8635992796376|
+-------+------------------+
```

Figure 2: Spark shell output showing Step 2 – data quality check

3.2 Feature Engineering

Two feature configurations are evaluated in this phase:

Table 2: Feature sets used in this phase

Feature Set	Features Included	Count
Full	month, week_no, district_indexed, prev_week_total + 31 crime-type columns	35
Reduced	month, week_no, district_indexed, prev_week_total only	4

The district_indexed column was produced by StringIndexer in Phase 2, encoding each district as a unique categorical index. The prev_week_total lag feature captures the previous week's total crime count per district and was also engineered in the data preprocessing phase using Spark Window functions. All null values were filled with zero before assembly.

```
scala> // ============================================
     | // STEP 3: Define Feature Columns
     | // ============================================
     |
     | val crimeTypeCols = Array(
     |   "ARSON", "ASSAULT", "BATTERY", "BURGLARY",
     |   "CONCEALED CARRY LICENSE VIOLATION", "CRIMINAL DAMAGE",
     |   "CRIMINAL SEXUAL ASSAULT", "CRIMINAL TRESPASS",
     |   "DECEPTIVE PRACTICE", "GAMBLING", "HOMICIDE",
     |   "HUMAN TRAFFICKING", "INTERFERENCE WITH PUBLIC OFFICER",
     |   "INTIMIDATION", "KIDNAPPING", "LIQUOR LAW VIOLATION",
     |   "MOTOR VEHICLE THEFT", "NARCOTICS", "NON-CRIMINAL",
     |   "OBSCENITY", "OFFENSE INVOLVING CHILDREN",
     |   "OTHER NARCOTIC VIOLATION", "OTHER OFFENSE",
     |   "PROSTITUTION", "PUBLIC INDECENCY",
     |   "PUBLIC PEACE VIOLATION", "ROBBERY", "SEX OFFENSE",
     |   "STALKING", "THEFT", "WEAPONS VIOLATION"
     | )
     |
     | val featureCols = Array(
     |   "month", "week_no", "district_indexed", "prev_week_total"
     | ) ++ crimeTypeCols
     |
     | println(s"Total feature columns: ${featureCols.length}")
     | featureCols.foreach(println)
Total feature columns: 35
month
week_no
district_indexed
prev_week_total
ARSON
ASSAULT
BATTERY
BURGLARY
CONCEALED CARRY LICENSE VIOLATION
CRIMINAL DAMAGE
CRIMINAL SEXUAL ASSAULT
CRIMINAL TRESPASS
DECEPTIVE PRACTICE
GAMBLING
HOMICIDE
HUMAN TRAFFICKING
INTERFERENCE WITH PUBLIC OFFICER
INTIMIDATION
KIDNAPPING
LIQUOR LAW VIOLATION
MOTOR VEHICLE THEFT
NARCOTICS
NON-CRIMINAL
OBSCENITY
OFFENSE INVOLVING CHILDREN
OTHER NARCOTIC VIOLATION
OTHER OFFENSE
PROSTITUTION
PUBLIC INDECENCY
PUBLIC PEACE VIOLATION
ROBBERY
SEX OFFENSE
STALKING
THEFT
WEAPONS VIOLATION
val crimeTypeCols: Array[String] = Array(ARSON, ASSAULT, BATTERY, BUR
DE, HUMAN TRAFFICKING, INTERFERENCE WITH PUBLIC OFFICER, INTIMIDATION
LATION, OTHER OFFENSE, PROSTITUTION, PUBLIC INDECENCY, PUBLIC PEACE V
val featureCols: Array[String] = Array(month, week_no, district_index
```

Figure 3: Spark shell output showing Step 3 – feature definition

```
scala> // ============================================
     | // STEP 4: Assemble Features
     | // Scaling is deferred until AFTER the split
     | // to prevent data leakage as planned in Phase 2
     | // ============================================
     |
     | val cleanDF = preprocessedDF.na.fill(0)
     |
     | val assembler = new VectorAssembler()
     |   .setInputCols(featureCols)
     |   .setOutputCol("features_raw")
     |
     | val assembledDF = assembler.transform(cleanDF)
     |
     | println("Feature assembly complete.")
     | println("Note: Scaling will be applied AFTER train/test split")
     | println("      to prevent data leakage from test set into scaler.")
     | assembledDF.select("district", "week_start", "total_crime_count")
     |   .show(3, truncate = false)
Feature assembly complete.
Note: Scaling will be applied AFTER train/test split
      to prevent data leakage from test set into scaler.
+--------+-------------------+-----------------+
|district|week_start         |total_crime_count|
+--------+-------------------+-----------------+
|1       |2021-01-04 00:00:00|123              |
|1       |2021-01-11 00:00:00|124              |
|1       |2021-01-18 00:00:00|118              |
+--------+-------------------+-----------------+
only showing top 3 rows
val cleanDF: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 36 more fields]
val assembler: org.apache.spark.ml.feature.VectorAssembler = VectorAssembler: uid=vecAssembler_b88d82b0fbc6,
=error, numInputCols=35
val assembledDF: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 37 more fields]
```

Figure 4: Spark shell output showing Step 4 – feature assembly

3.3 Train/Test Split

The dataset was split 70/30 using a fixed random seed of 42 for reproducibility. The split was applied to the assembled (unscaled) DataFrame to ensure that scaling statistics are derived only from training data.

Table 3: Train/test split statistics

Split	Rows	Percentage
Training set	3,312 rows	71.3%
Test set	1,332 rows	28.7%
Total	4,644 rows	100%

```
scala> // ============================================
     | // STEP 5: Train/Test Split (70/30, seed = 42)
     | // Split happens BEFORE scaling — this is critical
     | // to avoid data leakage
     | // ============================================
     |
     | val Array(trainRaw, testRaw) = assembledDF.randomSplit(Array(0.7, 0.3), seed = 42)
     |
     | println(s"Training set size: ${trainRaw.count()} rows")
     | println(s"Test set size:     ${testRaw.count()} rows")
     | println(s"Total:             ${trainRaw.count() + testRaw.count()} rows")
26/03/16 11:08:14 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can
justed by setting 'spark.sql.debug.maxToStringFields'.
Training set size: 3312 rows
Test set size:     1332 rows
Total:             4644 rows
val trainRaw: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [district: int, week_start: timestamp ... 37 more fields]
val testRaw: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [district: int, week_start: timestamp ... 37 more fields]
```

Figure 5: Spark shell output showing Step 5 – train/test split

3.4 StandardScaler

StandardScaler was applied after the train/test split to prevent data leakage. The scaler was fitted exclusively on the training set, computing mean and standard deviation from training data only. These training statistics were then applied to both the training and test sets. This is the methodology promised in data preprocessing phase under 'Planned Numerical Scaling and Strategic Deferral.'This approach ensures that high-magnitude features do not exert undue numerical dominance over the model, a best practice for linear algorithms [1].

```
scala> // ========================================
     | // STEP 6: Apply StandardScaler
     | // Fitted on TRAINING DATA ONLY
     | // Then applied to both train and test sets
     | // This fulfills the Phase 2 promise of
     | // methodological integrity and no data leakage
     | // ========================================
     |
     | val scaler = new StandardScaler()
     |   .setInputCol("features_raw")
     |   .setOutputCol("features")
     |   .setWithMean(true)
     |   .setWithStd(true)
     |
     | // FIT only on training data
     | val scalerModel = scaler.fit(trainRaw)
     |
     | // Transform both sets using training statistics
     | val trainDF = scalerModel.transform(trainRaw)
     | val testDF  = scalerModel.transform(testRaw)
     |
     | println("StandardScaler fitted on training data only.")
     | println("Scaler applied to both training and test sets.")
     | println(s"Training rows: ${trainDF.count()}")
     | println(s"Test rows:     ${testDF.count()}")
StandardScaler fitted on training data only.
Scaler applied to both training and test sets.
Training rows: 3312
Test rows:     1332
val scaler: org.apache.spark.ml.feature.StandardScaler = stdScal_9c0eb0cf1878
val scalerModel: org.apache.spark.ml.feature.StandardScalerModel = StandardScalerModel: uid=stdScal_9c0eb0cf1878, numFeatures=35, with
Mean=true, withStd=true
val trainDF: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 38 more fields]
val testDF: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 38 more fields]
```

Figure 6: Spark shell output showing Step 6 – StandardScaler

4. Baseline Model

The baseline model predicts the training set mean (204.85 crimes per week) for every district-week combination in the test set. This represents the simplest possible prediction strategy and establishes the performance floor that trained models must exceed to demonstrate genuine learning.

Code:

```
val trainMean = trainDF.select(mean("total_crime_count")).first().getDouble(0)
val baselineTestDF = testDF.withColumn("prediction", lit(trainMean))
```

```
scala> // ========================================
     | // STEP 7: Baseline Model (Mean Prediction)
     | // Predicts the training mean for every week
     | // Serves as the minimum performance threshold
     | // ========================================
     |
     | val trainMean = trainDF
     |   .select(mean("total_crime_count"))
     |   .first()
     |   .getDouble(0)
     |
     | println(s"Training set mean crime count: $trainMean")
     |
     | val baselineTestDF = testDF
     |   .withColumn("prediction", lit(trainMean))
     |
     | val evaluatorRMSE = new RegressionEvaluator()
     |   .setLabelCol("total_crime_count")
     |   .setPredictionCol("prediction")
     |   .setMetricName("rmse")
     |
     | val evaluatorMAE = new RegressionEvaluator()
     |   .setLabelCol("total_crime_count")
     |   .setPredictionCol("prediction")
     |   .setMetricName("mae")
     |
     | val evaluatorR2 = new RegressionEvaluator()
     |   .setLabelCol("total_crime_count")
     |   .setPredictionCol("prediction")
     |   .setMetricName("r2")
     |
     | val baselineRMSE = evaluatorRMSE.evaluate(baselineTestDF)
     | val baselineMAE  = evaluatorMAE.evaluate(baselineTestDF)
     | val baselineR2   = evaluatorR2.evaluate(baselineTestDF)
     |
     | println(s"Baseline RMSE: $baselineRMSE")
     | println(s"Baseline MAE:  $baselineMAE")
     | println(s"Baseline R2:   $baselineR2")
Training set mean crime count: 204.85175120772948
Baseline RMSE: 68.6117480147915
Baseline MAE:  54.42845200272755
Baseline R2:   -1.8921184344877773E-5
val trainMean: Double = 204.85175120772948
val baselineTestDF: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 39 more fields]
val evaluatorRMSE: org.apache.spark.ml.evaluation.RegressionEvaluator = RegressionEvaluator: uid=regEval_db08d33e73bb, metricName=rmse
, throughOrigin=false
val evaluatorMAE: org.apache.spark.ml.evaluation.RegressionEvaluator = RegressionEvaluator: uid=regEval_c17b6998a0fd, metricName=mae,
throughOrigin=false
val evaluatorR2: org.apache.spark.ml.evaluation.RegressionEvaluator = RegressionEvaluator: uid=regEval_85b0e7e8d0ee, metricName=r2, th
roughOrigin=false
val baselineRMSE: Double ...
```

Figure 7: Spark shell output showing Step 7 – baseline metrics

Table 4: Baseline model metrics

Metric	Value	Interpretation
Training mean	204.85 crimes/week	Predicted value for every test row
RMSE	68.61	Average prediction error of 68.61 crimes per week
MAE	54.43	Median prediction error of 54.43 crimes per week
R^2	≈ 0.00	Model explains 0% of variance — random-guess level

The baseline R^2 of approximately zero confirms that simply predicting the mean provides no useful forecasting signal. All trained models in this phase must achieve substantially lower RMSE and higher R^2 to be considered meaningful.

5. Full Feature Set Models (35 Features)

The following three models were trained using all 35 available features including the 31 individual crime-type pivot columns. An important limitation of this configuration is discussed after the results. The selection of Linear Regression, Decision Tree, and Random Forest regressors was driven by the need to compare baseline linear relationships against non-linear, hierarchical patterns. Linear Regression serves as a traditional statistical benchmark for continuous count forecasting. Decision Trees were chosen for their ability to handle non-linearities and categorical district data without requiring feature scaling. Finally, Random Forest was selected as the primary candidate because its ensemble logic (averaging multiple independent trees) is specifically designed to reduce model variance and provide higher generalization accuracy on volatile datasets like urban crime [1].

5.1 Linear Regression

Linear Regression was trained with L2 regularization (regParam = 0.1) and a maximum of 100 iterations. The model learns a weighted linear combination of all 35 scaled features to minimize the squared prediction error.

Code:

```
val lr = new LinearRegression()
  .setLabelCol("total_crime_count")
  .setFeaturesCol("features")
  .setMaxIter(100)
  .setRegParam(0.1)
  .setElasticNetParam(0.0)
val lrModel = lr.fit(trainDF)
val lrPredictions = lrModel.transform(testDF)
```

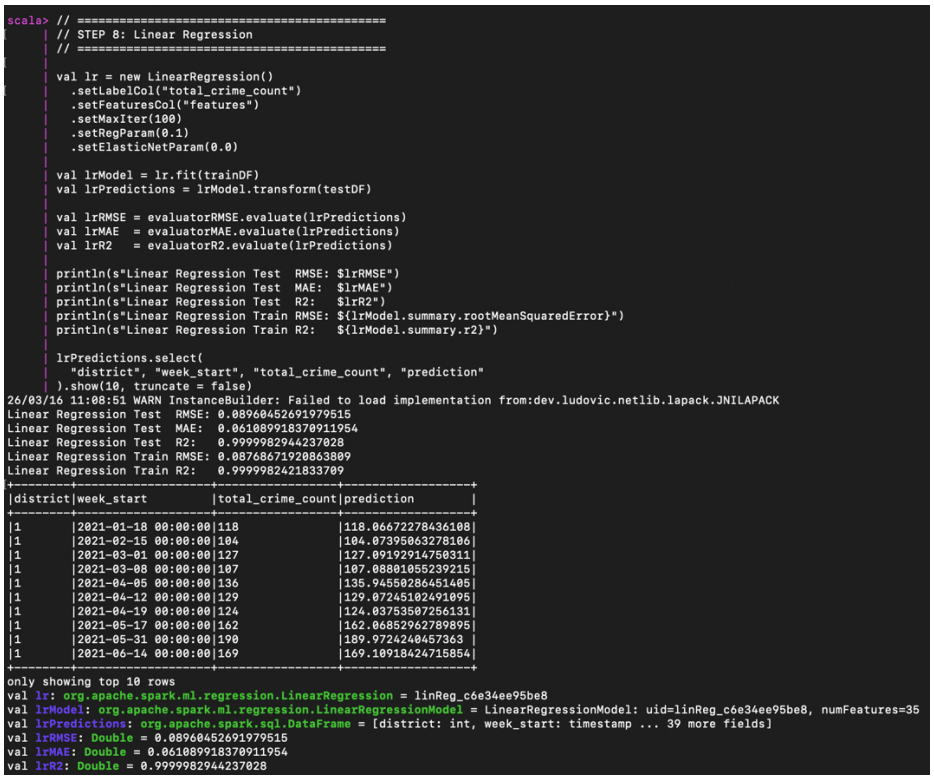

Figure 8: Spark shell output showing Step 8 – Linear Regression

Table 5: Linear Regression metrics (full features)

Metric	Training	Test
RMSE	0.0877	0.0896
MAE	N/A	0.0611
R^2	0.9999982	0.9999983

Sample predictions (District 1, selected weeks):

district	week_start	actual	predicted
1	2021-01-18	118	118.07
1	2021-02-15	104	104.07
1	2021-03-01	127	127.09
1	2021-05-31	190	189.97

The near-perfect R^2 of 0.9999983 on the test set is the result of a structural property of the dataset: total_crime_count is mathematically equal to the sum of all individual crime-type columns included as features. The Linear Regression model learns this additive identity rather than a genuinely predictive relationship. This is addressed in Section 6.

5.2 Decision Tree Regressor

The Decision Tree was trained with a maximum depth of 5 and a fixed seed of 42. Unlike Linear Regression, tree-based models split on feature thresholds rather than learning linear combinations, making them less susceptible to the additive identity problem.

Code:

```
val dt = new DecisionTreeRegressor()
  .setLabelCol("total_crime_count")
  .setFeaturesCol("features")
  .setMaxDepth(5)
  .setSeed(42)
val dtModel = dt.fit(trainDF)
val dtPredictions = dtModel.transform(testDF)
```

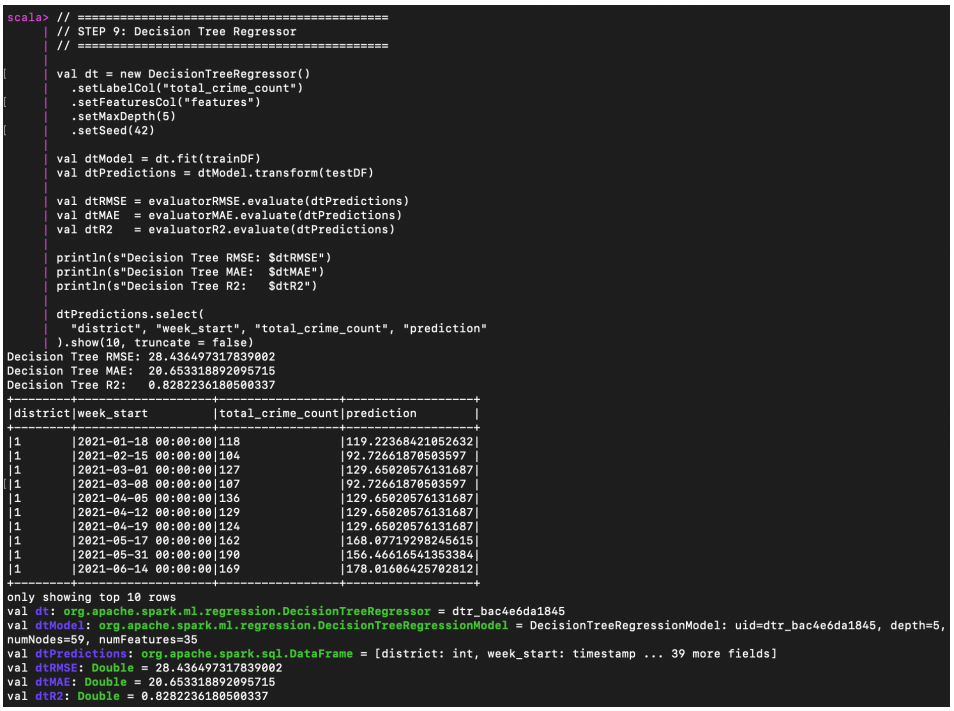

Figure 9: Spark shell output showing Step 9 – Decision Tree

Table 6: Decision Tree metrics (full features)

Metric	Test Value	vs Baseline
RMSE	28.44	58.6% reduction from 68.61
MAE	20.65	62.1% reduction from 54.43
R^2	0.8282	Explains 82.8% of variance

The Decision Tree achieves $R^2 = 0.828$, meaning it correctly explains 82.8% of the week-to-week variation in crime counts across all districts. In practical terms, the model's predictions are wrong by an average of 28 crimes per week, for example, if a district actually had 200 crimes in a given week, the model would typically predict somewhere between 172 and 228. This is a significant improvement over the baseline, which was wrong by an average of 68 crimes per week. The tree reached a depth of 5 with 59 decision nodes, meaning it learned 59 specific rules from the training data to make its predictions.

5.3 Random Forest Regressor

The Random Forest was trained with 50 trees, a maximum depth of 5, and a fixed seed of 42. By averaging predictions across 50 independently trained decision trees, the ensemble reduces overfitting and variance compared to a single tree. We employed this ensemble approach to reduce model variance and improve generalization [1].

Code:

```
val rf = new RandomForestRegressor()
  .setLabelCol("total_crime_count")
  .setFeaturesCol("features")
  .setNumTrees(50)
  .setMaxDepth(5)
  .setSeed(42)
val rfModel = rf.fit(trainDF)
val rfPredictions = rfModel.transform(testDF)
```

```
scala> // ========================================
     | // STEP 10: Random Forest Regressor
     | // ========================================
     |
     | val rf = new RandomForestRegressor()
     |   .setLabelCol("total_crime_count")
     |   .setFeaturesCol("features")
     |   .setNumTrees(50)
     |   .setMaxDepth(5)
     |   .setSeed(42)
     |
     | val rfModel = rf.fit(trainDF)
     | val rfPredictions = rfModel.transform(testDF)
     |
     | val rfRMSE = evaluatorRMSE.evaluate(rfPredictions)
     | val rfMAE  = evaluatorMAE.evaluate(rfPredictions)
     | val rfR2   = evaluatorR2.evaluate(rfPredictions)
     |
     | println(s"Random Forest RMSE: $rfRMSE")
     | println(s"Random Forest MAE:  $rfMAE")
     | println(s"Random Forest R2:   $rfR2")
     |
     | rfPredictions.select(
     |   "district", "week_start", "total_crime_count", "prediction"
     | ).show(10, truncate = false)
Random Forest RMSE: 21.06641053317371
Random Forest MAE:  14.827998536461829
Random Forest R2:   0.9057259128288446
+--------+-------------------+-----------------+------------------+
|district|week_start         |total_crime_count|prediction        |
+--------+-------------------+-----------------+------------------+
|1       |2021-01-18 00:00:00|118              |120.54325191525844|
|1       |2021-02-15 00:00:00|104              |103.86101951279174|
|1       |2021-03-01 00:00:00|127              |142.82791730361117|
|1       |2021-03-08 00:00:00|107              |116.59331686808028|
|1       |2021-04-05 00:00:00|136              |133.50819754934145|
|1       |2021-04-12 00:00:00|129              |134.20019055690528|
|1       |2021-04-19 00:00:00|124              |137.30928712244352|
|1       |2021-05-17 00:00:00|162              |169.35590200448746|
|1       |2021-05-31 00:00:00|190              |174.69277988819903|
|1       |2021-06-14 00:00:00|169              |180.35517310767912|
+--------+-------------------+-----------------+------------------+
only showing top 10 rows
val rf: org.apache.spark.ml.regression.RandomForestRegressor = rfr_9b444092e4dd
val rfModel: org.apache.spark.ml.regression.RandomForestRegressionModel = RandomForestRegressionModel: uid=rfr_9b444092e4dd, numTrees=
50, numFeatures=35
val rfPredictions: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 39 more fields]
val rfRMSE: Double = 21.06641053317371
val rfMAE: Double = 14.827998536461829
val rfR2: Double = 0.9057259128288446
```

Figure 10: Spark shell output showing Step 10 – Random Forest

Table 7: Random Forest metrics (full features)

Metric	Test Value	vs Baseline
RMSE	21.07	69.3% reduction from 68.61
MAE	14.83	72.8% reduction from 54.43
R^2	0.9057	Explains 90.6% of variance

The Random Forest achieves the best performance among full-feature models with $R^2 = 0.906$ and RMSE = 21.07. The ensemble approach reduces prediction error by 69.3% compared to the baseline. Predictions are on average within 21 crimes per week of actual counts.

5.4 Model Comparison — Full Features

```
scala> // ============================================
     | // STEP 11: Model Comparison Summary
     | // ============================================
     |
     | println("=" * 65)
     | println(f"${"Model"}%-25s ${"RMSE"}%10s ${"MAE"}%10s ${"R2"}%10s")
     | println("=" * 65)
     | println(f"${"Baseline (Mean)"}%-25s ${baselineRMSE}%10.4f ${baselineMAE}%10.4f ${baselineR2}%10.4f")
     | println(f"${"Linear Regression"}%-25s ${lrRMSE}%10.4f ${lrMAE}%10.4f ${lrR2}%10.4f")
     | println(f"${"Decision Tree"}%-25s ${dtRMSE}%10.4f ${dtMAE}%10.4f ${dtR2}%10.4f")
     | println(f"${"Random Forest"}%-25s ${rfRMSE}%10.4f ${rfMAE}%10.4f ${rfR2}%10.4f")
     | println("=" * 65)
=================================================================
Model                           RMSE        MAE         R2
=================================================================
Baseline (Mean)              68.6117    54.4285    -0.0000
Linear Regression             0.0896     0.0611     1.0000
Decision Tree                28.4365    20.6533     0.8282
Random Forest                21.0664    14.8280     0.9057
=================================================================
```

Figure 11: Spark shell output showing Step 11 – model comparison

Table 8: Model comparison — full feature set

Model	RMSE	MAE	R^2	Notes
Baseline (Mean)	68.6117	54.4285	≈0.0000	Performance floor
Linear Regression*	0.0896	0.0611	0.9999983	Identity artifact
Decision Tree	28.4365	20.6533	0.8282	Genuine learning
Random Forest	21.0664	14.8280	0.9057	Best full-feature model

The Linear Regression result is flagged(*) because it exploits the mathematical identity between total_crime_count and the sum of crime-type columns. Decision Tree and Random Forest are not affected by this because tree splits are based on inequality thresholds, not linear sums. The reduced feature set evaluation in Section 6 provides the honest forecasting assessment.

5.5 Feature Importance — Full Random Forest

```
scala> // ========================================
     | // STEP 12: Feature Importance (Random Forest)
     | // ========================================
     |
     | val featureImportances = rfModel.featureImportances
     | val importanceWithNames = featureCols
     |   .zip(featureImportances.toArray)
     |   .sortBy(-_._2)
     |
     | println("Top 15 Most Important Features (Random Forest):")
     | println(f"${"Feature"}%-40s ${"Importance"}%10s")
     | println("-" * 55)
     | importanceWithNames.take(15).foreach { case (name, importance) =>
     |   println(f"${name}%-40s ${importance}%10.6f")
     | }
Top 15 Most Important Features (Random Forest):
Feature                                  Importance
-------------------------------------------------------
prev_week_total                            0.394634
BATTERY                                    0.194011
CRIMINAL DAMAGE                            0.100807
MOTOR VEHICLE THEFT                        0.093564
THEFT                                      0.087717
ROBBERY                                    0.036065
ASSAULT                                    0.025407
DECEPTIVE PRACTICE                         0.014860
district_indexed                           0.013667
OTHER OFFENSE                              0.011706
WEAPONS VIOLATION                          0.007078
CRIMINAL TRESPASS                          0.005900
BURGLARY                                   0.005767
SEX OFFENSE                                0.002366
NARCOTICS                                  0.001269
val featureImportances: org.apache.spark.ml.linalg.Vector = (35,[0,1,2,3,4,5,6,7,8,9,10,11,12,14,16,17,19,20,21,24,26,27,29,30,31,32,33,34],[2.7830857917236074E-4,0.0012083984662921716,0.013667364911806121,0.3946336385688242,4.696188080423905E-5,0.025406945455261898,0.19401070159163974,0.005766952624252625,2.22059995447832E-10,0.10080710100118394,2.933196464925019E-4,0.0058996232114202785,0.014860442085916485,5.1823589349352233E-5,9.251305750635437E-4,3.0235198805130023E-5,9.030199523471756E-4,0.09356364367463095,0.001269226271763550 5,4.40003389536096E-4,0.011706097180126109,3.4187321274989287E-4,4.288341643488068E-4,0.036065136718896884,0.0023664294745349085,2.340135149099884 6E-4,0.08771712449455016,0.00707765034326074 1])
val importanceWithNames: ...
```

Figure 12: Spark shell output showing Step 12 – feature importances

Table 9: Top 15 feature importances — full Random Forest

Rank	Feature	Importance
1	prev_week_total	0.394634 (39.5%)
2	BATTERY	0.194011 (19.4%)
3	CRIMINAL DAMAGE	0.100807 (10.1%)
4	MOTOR VEHICLE THEFT	0.093564 (9.4%)
5	THEFT	0.087717 (8.8%)
6	ROBBERY	0.036065 (3.6%)
7	ASSAULT	0.025407 (2.5%)
8	DECEPTIVE PRACTICE	0.014860 (1.5%)
9	district_indexed	0.013667 (1.4%)
10	OTHER OFFENSE	0.011706 (1.2%)

Even in the full feature set, prev_week_total is the single most important feature at 39.5%, confirming that last week's crime count carries more predictive weight than any individual crime-type column. BATTERY (19.4%) and CRIMINAL DAMAGE (10.1%) are the most influential crime-type features, consistent with them being among the highest-volume categories identified in the RDD Phase.

6. Reduced Feature Set Models (4 Features)

6.1 Motivation for Reduced Feature Set

The full feature set includes 31 individual crime-type columns that collectively sum to total_crime_count. This creates a mathematical identity: total = THEFT + BATTERY + ROBBERY + ... + all other types. Linear Regression exploits this relationship and achieves near-perfect accuracy by learning the identity function rather than a genuine temporal forecast.

To produce a meaningful evaluation of the model's actual forecasting ability, a reduced feature set was constructed using only the 4 features that represent genuine temporal, spatial, and momentum signals:

Feature	Type	Description
month	Temporal	Month of year (1–12) — captures seasonal patterns
week_no	Temporal	Week number (1–52) — captures intra-year cycles
district_indexed	Spatial	Encoded district category — captures district baseline
prev_week_total	Lag	Previous week's total crimes — captures momentum

With these 4 features, the model must predict this week's crime count without knowing the current week's crime-type breakdown, which is exactly the real-world forecasting scenario where future crime types are unknown in advance.

```
scala> // ============================================
     | // STEP 14: Reduced Feature Set
     | // Only temporal + spatial + lag features
     | // Excludes crime-type columns to avoid
     | // the mathematical identity problem
     | // ============================================
     |
     | val reducedFeatureCols = Array(
     |   "month", "week_no", "district_indexed", "prev_week_total"
     | )
     |
     | println("Reduced feature set (4 features only):")
     | reducedFeatureCols.foreach(println)
     | println(s"\nTotal reduced features: ${reducedFeatureCols.length}")
Reduced feature set (4 features only):
month
week_no
district_indexed
prev_week_total

Total reduced features: 4
val reducedFeatureCols: Array[String] = Array(month, week_no, district_indexed, prev_week_total)

scala>

scala>

scala> // ============================================
     | // STEP 15: Assemble + Split + Scale
     | // Reduced Feature Set
     | // ============================================
     |
     | val assemblerReduced = new VectorAssembler()
     |   .setInputCols(reducedFeatureCols)
     |   .setOutputCol("features_raw_reduced")
     |
     | val assembledReducedDF = assemblerReduced.transform(cleanDF)
     |
     | // Split FIRST
     | val Array(trainReducedRaw, testReducedRaw) =
     |   assembledReducedDF.randomSplit(Array(0.7, 0.3), seed = 42)
     |
     | // Fit scaler on training data only
     | val scalerReduced = new StandardScaler()
     |   .setInputCol("features_raw_reduced")
     |   .setOutputCol("features_reduced")
     |   .setWithMean(true)
     |   .setWithStd(true)
     |
     | val scalerReducedModel = scalerReduced.fit(trainReducedRaw)
     | val trainReducedDF = scalerReducedModel.transform(trainReducedRaw)
     | val testReducedDF  = scalerReducedModel.transform(testReducedRaw)
     |
     | println(s"Reduced training rows: ${trainReducedDF.count()}")
     | println(s"Reduced test rows:     ${testReducedDF.count()}")
Reduced training rows: 3312
Reduced test rows:     1332
```

Figure 13: Spark shell output showing Step 14/15 – reduced pipeline

6.2 Model Results — Reduced Features

All three models were retrained on the 4-feature reduced set using the same 70/30 split, seed = 42, and StandardScaler fitted on training data only.

Table 10: Model comparison — reduced feature set

Model	RMSE	MAE	R^2	vs Baseline
Baseline (Mean)	68.61	54.43	≈0.00	—
Linear Regression	29.05	20.98	0.8208	57.6% RMSE reduction
Decision Tree	29.19	21.35	0.8190	57.4% RMSE reduction
Random Forest	28.54	21.28	0.8269	58.4% RMSE reduction

All three models achieve R^2 between 0.819 and 0.827, explaining approximately 82% of the variance in weekly crime counts using only 4 features. The three models perform very similarly, which suggests the problem is well-conditioned and that the 4-feature set captures the dominant predictive signals. Random Forest leads with $R^2 = 0.827$ and RMSE = 28.54.

6.3 Feature Importance — Reduced Random Forest

```
     |
     | val rfReducedImportances = rfReducedModel.featureImportances
     | val reducedImportanceWithNames = reducedFeatureCols
     |   .zip(rfReducedImportances.toArray)
     |   .sortBy(-_._2)
     |
     | println("Feature Importance (Reduced Random Forest):")
     | println(f"${"Feature"}%-25s ${"Importance"}%10s")
     | println("-" * 40)
     | reducedImportanceWithNames.foreach { case (name, importance) =>
     |   println(f"${name}%-25s ${importance}%10.6f")
     | }
Feature Importance (Reduced Random Forest):
Feature                   Importance
----------------------------------------
prev_week_total             0.829129
district_indexed            0.127447
week_no                     0.030178
month                       0.013247
val rfReducedImportances: org.apache.spark.ml.linalg.Vector = (4,[0,1,2,3],[0.01324650
832,0.8291287349949177])
val reducedImportanceWithNames: Array[(String, Double)] = Array((prev_week_total,0.829
96832), (week_no,0.030177896339742195), (month,0.013246500172371858))
```

Figure 14: Spark shell output showing Step 17 – reduced feature importance

Table 11: Feature importances — reduced Random Forest

Rank	Feature	Importance	Interpretation
1	prev_week_total	0.829129 (82.9%)	Last week's count is dominant predictor
2	district_indexed	0.127447 (12.7%)	Each district has a unique crime level
3	week_no	0.030178 (3.0%)	Within-year weekly cycles
4	month	0.013247 (1.3%)	Seasonal monthly patterns

The reduced model reveals that prev_week_total alone accounts for 82.9% of the model's decisions. This powerfully confirms the preprocessing phase design decision to engineer a lag feature — historical momentum is the strongest available signal for short-term crime forecasting. district_indexed at 12.7% confirms that each district has a meaningfully different crime baseline. week_no and month together contribute only 4.3%, suggesting seasonal signals are secondary to recent momentum.

6.4 Full Comparison Table

```
     | println(f"${"RF — Reduced Features"}%-35s ${rfReducedRMSE}%10.4f ${rfReducedMAE}%10.4f ${rfReducedR2}%10.4f")
     | println("=" * 75)
LR Reduced RMSE: 29.045191996284945
LR Reduced MAE:  20.984033174664535
LR Reduced R2:   0.8207910263279232
DT Reduced RMSE: 29.18853658401931
DT Reduced MAE:  21.349374361299244
DT Reduced R2:   0.8190177878472602
RF Reduced RMSE: 28.542894369426964
RF Reduced MAE:  21.279497302127364
RF Reduced R2:   0.8269357877142312

===========================================================================
Model                                RMSE        MAE         R2
===========================================================================
Baseline (Mean)                     68.6117    54.4285    -0.0000
--- Full Feature Set (35 features) ---
LR — Full Features                   0.0896     0.0611     1.0000
DT — Full Features                  28.4365    20.6533     0.8282
RF — Full Features                  21.0664    14.8280     0.9057
--- Reduced Feature Set (4 features) ---
LR — Reduced Features               29.0452    20.9840     0.8208
DT — Reduced Features               29.1885    21.3494     0.8190
RF — Reduced Features               28.5429    21.2795     0.8269
===========================================================================
val lrReduced: org.apache.spark.ml.regression.LinearRegression = linReg_fc30cbeda999
val lrReducedModel: org.apache.spark.ml.regression.LinearRegressionModel = LinearRegressionModel: uid=linReg_fc30cbeda999, numFeatures=4
val lrReducedPred: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 39 more fields]
val lrReducedRMSE: Double = 29.045191996284945
val lrReducedMAE: Double = 20.984033174664535
val lrReducedR2: Double = 0.8207910263279232
val dtReduced: org.apache.spark.ml.regression.DecisionTreeRegressor = dtr_b8aca5e6d5a9
val dtReducedModel: org.apache.spark...
```

Figure 15: Spark shell output showing Step 16 – full comparison table

Table 12: Full model comparison across all configurations

Model	RMSE	MAE	R^2
Baseline (Mean)	68.6117	54.4285	≈0.0000
— Full Feature Set (35 features) —			
LR – Full Features*	0.0896	0.0611	0.9999983
DT – Full Features	28.4365	20.6533	0.8282
RF – Full Features	21.0664	14.8280	0.9057
— Reduced Feature Set (4 features) —			
LR – Reduced Features	29.0452	20.9840	0.8208
DT – Reduced Features	29.1885	21.3494	0.8190
RF – Reduced Features	28.5429	21.2795	0.8269

7. Analysis and Interpretation

7.1 Model Performance Analysis

The reduced feature set Random Forest ($R^2 = 0.827$, RMSE = 28.54) is the primary forecasting result of this phase. Using only 4 features — prev_week_total, district_indexed, week_no, and month — the model predicts weekly district-level crime counts within approximately 28–29 crimes on average. reducing prediction error by 58.4% compared to the mean baseline.

Model performance was assessed using Root Mean Squared Error (RMSE) to quantify the average prediction deviation in original units [2].

The convergence of all three reduced-feature models to similar performance (R^2 between 0.819 and 0.827) indicates that the 4-feature set captures most of the available forecasting signal, and that additional model complexity beyond this does not yield significant gains. The similarity in performance also suggests the problem has a natural performance ceiling given the available features, which is a realistic outcome for short-term crime forecasting.

The full feature Random Forest ($R^2 = 0.906$, RMSE = 21.07) performs better than the reduced models, but this improvement is driven by using the same week's crime-type counts as input features — data that would only be available after the week has already ended. In a genuine forecasting scenario where the goal is to predict next week's crime count before it happens, only the reduced feature set reflects what would realistically be known in advance.The reduced feature results therefore represent the more operationally valid evaluation.

7.2 Feature Importance Insights

The feature importance analysis across both feature configurations confirms several findings from the exploratory phases. prev_week_total is the dominant predictor in both configurations — accounting for 39.5% of importance in the full model and 82.9% in the reduced model. This validates the preprocessing phase decision to engineer a lag feature as the primary forecasting signal.

district_indexed ranks second in the reduced model at 12.7%, confirming that each district has a distinct crime-level baseline that the model must account for separately. This is consistent with the Phase 3 RDD finding that crime is highly unequally distributed across districts, with District 8 recording more than three times the crimes of District 20.

In the full feature model, BATTERY (19.4%), CRIMINAL DAMAGE (10.1%), MOTOR VEHICLE THEFT (9.4%), and THEFT (8.8%) are the most influential crime-type features. These four categories account for 48.6% of the model's feature importance, suggesting that the weekly volume of these specific crime types is particularly predictive of total weekly crime counts.

7.3 Limitations and Future Work

The primary limitation of the current approach is that the feature set does not include external contextual variables such as weather, public events, school schedules, or economic indicators that may influence weekly crime volumes. Including such variables could potentially improve model performance beyond the current R^2 ceiling of approximately 0.83.

A second limitation is that the random 70/30 split mixes records from different years rather than using a strict temporal split. A more rigorous evaluation would train on 2021–2023 and test exclusively on 2024, ensuring that the model is evaluated on genuinely future data it has never seen.

Finally, the current model predicts total crime count as a single number per district per week. A more granular approach would forecast individual crime-type volumes separately, enabling more targeted resource deployment by crime category.

Future improvements could include: incorporating weather data as an external feature, applying a strict temporal train/test split, experimenting with Gradient Boosted Trees for improved accuracy, and extending predictions to individual crime categories rather than aggregate totals.

8. Reference

[1] J. S. Damji, B. Wenig, T. Das, and D. Lee, Learning Spark: Lightning-Fast Data Analytics, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020. [Online]. Available: https://vdoc.pub/download/learning-spark-lightning-fast-data-analytics-61bd73j55np0.

[2] Apache Software Foundation, "Spark MLlib Guide," Apache Spark Documentation, 2026. [Online]. Available: https://spark.apache.org/docs/latest/api/sql/index.html [Accessed: 16-Mar-2026].